

# Amélioration des performances I/O de l'application HPC Gysela



ComPas 2026

Alexandre Lou-Poueyou

Encadrants: Francieli Boito, Méline Trochon, Luan Teylo



# Plan

1. Introduction
2. Concepts clés
3. Méthodologie
4. Résultats
5. Conclusion

# 01 Introduction





# Gysela — Simulation gyrocinétique de la turbulence plasma

## Contexte : fusion nucléaire

- ▶ Les tokamaks confinent un plasma à très haute température
- ▶ La turbulence plasma dégrade le confinement
- ▶ Gysela : code de simulation gyrocinétique développé par le CEA pour étudier cette turbulence
- ▶ Modèle de Vlasov gyrocinétique 5D :  
 $f(r, \theta, \varphi, v_{\parallel}, \mu)$

## L'écriture des checkpoints présente un problème

- ▶ Les checkpoints en production sont volumineux et bloquent la simulation
- ▶ Variabilité jusqu'à  $\times 3$  sur un système de fichier parallèle (PFS) partagé en production
  - Trouver une meilleure stratégie I/O est critique
- ▶ Volume de données très important à écrire



# Problématique

## Notre objectif

- ▶ Améliorer les performances d'écriture des checkpoints
- ▶ Étudier l'impact de plusieurs paramètres I/O
- ▶ Trouver la meilleure stratégie I/O pour Gysela en contexte HPC

## 02

# Concepts clés





# Système de fichiers parallèles – PFS

## Object Storage Target – OST

- ▶ Unité de stockage logique ou physique
  - Serveur de stockage
  - groupe de disques RAID
- ▶ Stockage des données sous forme d'objets
- ▶ Permet un découpage des données sur plusieurs système de stockage

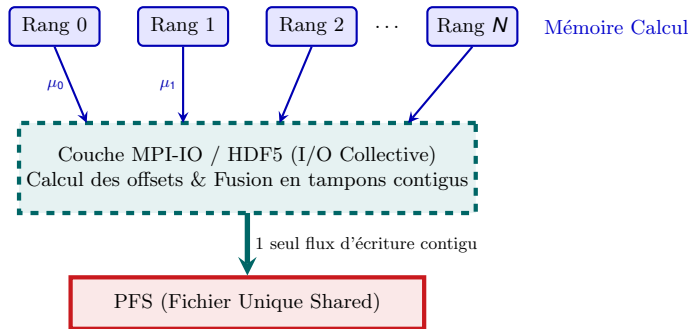
## Le striping

- ▶ Technique de distribution de données
  - Découpage d'un fichier en blocs (stripes)
  - Repartition de ces blocs en parallèle sur les OST
- ▶ Paramètre clés configurables:
  - Stripe Count: nombre d'OST utilisés (ex: 1 à 8)
  - Stripe Size: taille de chaque bloc de données écrit dans un round-robin
- ▶ Objectif : Augmenter l'usage de la bande passante



## Single Shared File — SSF

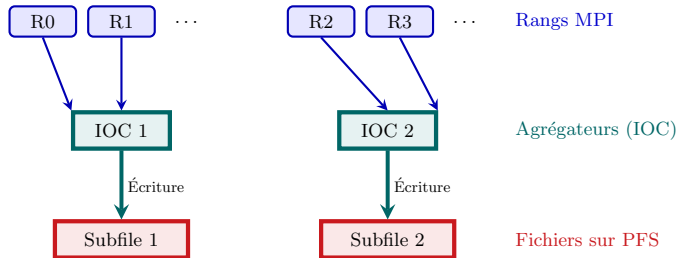
- ▶ Tous les rang MPI écrivent en parallèle dans un fichier commun.
- ▶ Accès non contigus calculés par HDF5/MPI-IO
- ▶ Aggrège toutes les données avant de les envoyer au PFS





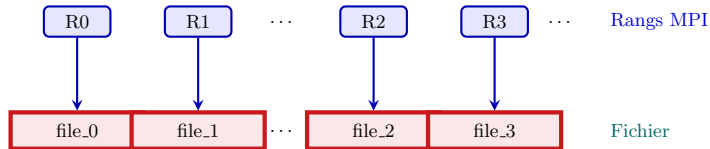
# Subfiling

- ▶ Les rangs MPI sont divisés en plusieurs groupes (sous-ensembles).
- ▶ Chaque groupe élit un ou plusieurs I/O Concentrators (IOC) / Agrégateurs.
- ▶ L'application écrit un nombre fixe de fichiers intermédiaire régent par un fichier de configuration



## File Per Process — FPP

- ▶ Chaque rang MPI gère son unique portion de données
- ▶ Chaque rang crée, écrit et ferme son fichier indépendamment des autres
- ▶ Génère autant de fichier que de processus MPI



# 03

## Méthodologie





# Outils et approche expérimentale

## **gys\_io — Noyau I/O de Gysela**

- ▶ Reproduit le pattern d'écriture des checkpoints de Gysela
- ▶ Benchmarks rapides et reproductibles
- ▶ Grille 5D :  $128 \times 128 \times 64 \times 64 \times 64$

## **PDI — Configuration des stratégies I/O**

- ▶ Stratégies I/O changées via fichier YAML
- ▶ Single Shared File (SSF), Subfiling, File Per Process (FPP)

## **IOPS — Automatisation des benchmarks**

- ▶ Balayage paramétrique et soumission SLURM
- ▶ Extraction : `time_write` (s), `time_total` (s)

## **Plafrim — Première étude de cas**

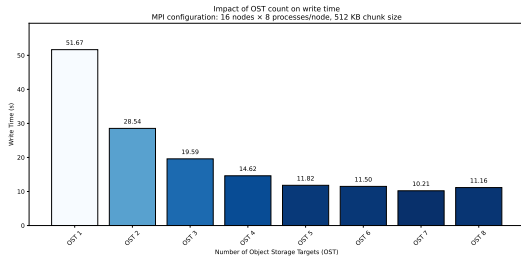
- ▶ PFS BeegFS, jusqu'à 8 OST
- ▶ Jusqu'à 44 nœuds **bora** disponibles
- ▶ 5 répétitions par configuration, allocation exclusive

# 04 Résultats





## Configuration PFS — Impact du nombre d'OST sur le temps d'écriture



[Figure : temps d'écriture (s) en fonction du nombre d'OST, 1 à 8]

### Observation

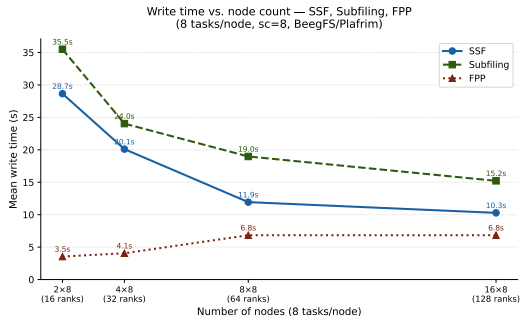
- ▶ Gain de presque  $\times 5$  : 1 OST  $\rightarrow$  8 OST
- ▶ Progression quasi-linéaire jusqu'à 5 OST
- ▶ Plateau au-delà de 5-7 OST

### À retenir

Utilisation de 8 OST pour tous les tests suivants.



## Scalabilité MPI — Toutes stratégies (8 tâches/nœud, 8 OST)



[Figure : temps d'écriture (s) vs nœuds — 3 courbes : SSF, Subfiling, FPP]

### SSF

Scalabilité quasi-linéaire — meilleur à grande échelle

### Subfiling

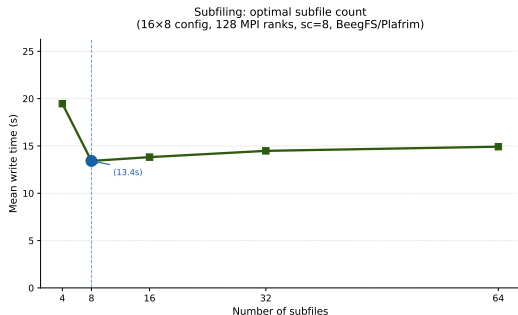
S'améliore positivement — se stabilise à 8–16 nœuds

### FPP

Dégradation très légère avec le nombre de nœuds mais meilleures performances dans toutes les configurations



## Subfiling — Nombre optimal de subfiles et modification du code



[Figure : temps d'écriture (s) vs nombre de subfiles, config 16×8]

### Observation

- ▶ Optimal à 8 subfiles = nb OST
- ▶ Confirme : `nb_subfiles = nb_OST`
- ▶ Différence 8 vs 16 : <3%

### Modification du code requise

Le Subfiling nécessite

`MPI_THREAD_MULTIPLE`

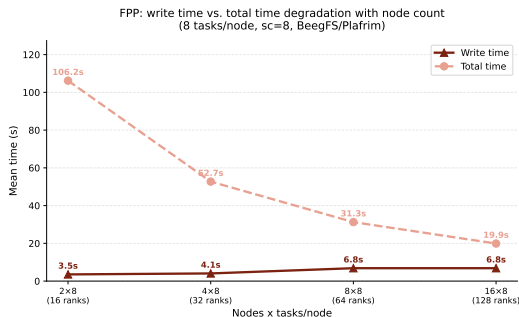
```
MPI_Init() →  
MPI_Init_thread(...,  
MPI_THREAD_MULTIPLE, ...)
```

Nécessaire pour les appels MPI des  
threads IOC





## FPP — Bonnes performances constantes



[Figure : temps d'écriture et temps total (s) vs  
nœuds — FPP uniquement]

### Performances peak

Meilleures performances I/O avec  
une très légère dégradation qui se  
stabilise à 8 nœuds

### Le vrai compromis

FPP ne nécessite pas de topologie  
particulière pour ses meilleures  
performances I/O.

Gysela en production requiert  
beaucoup de nœuds – heureusement  
les performances restent stables entre  
petites et grandes configurations.

# 05 Conclusion





## Meilleures configurations et perspectives

### Recommandations — Plafrim/BeegFS

- ▶ Toujours utiliser 8 OST — gain  $\times 5$  gratuit
- ▶ FPP recommandé à cette échelle de grille
- ▶ Subfiling : fixer `nb_subfiles = nb_OST`

### À confirmer

- ▶ Plus d'OST améliorent-ils les performances Subfiling ?
- ▶ Est ce que la stripe size a t elle un impact concret à grande échelle ?

## Perspectives

- ▶ Montée en échelle sur Adastral et Irène
- ▶ Weak scaling — grille 5D proportionnelle aux rangs
- ▶ I/O Concentrators (`aggregator_count`) — impact sur Subfiling
- ▶ IOPS + Optimisation bayésienne — recherche automatique de configuration optimale
- ▶ VeloC — Utilisation de VeloC pour essayer de remplacer HDF5
- ▶ GPU I/O — Explorer les possibilités de GPU direct I/O

Merci.

